AGENTIC AI · PORTFOLIO PROJECT BLUEPRINT

AutoResolve AI

An Autonomous Multi-Agent Customer Support & Ops Resolution System
with Adversarial Testing, Prompt-Injection Defense, and Self-Improving Knowledge

LLM Orchestration | RAG | Tool / Function Calling | Multi-Agent Systems | Agent Evaluation | Guardrails & Security | Production Deployment

Prepared as a complete build guide — from first prototype to production-grade system.

§ Contents

01 Project Overview —

02 The Real-World Problem —

03 Why This Project Is Valuable for Your Resume —

04 Agentic AI Concepts You Will Learn —

05 Recommended Tech Stack —

06 System Architecture —

07 Agents & Responsibilities INCL. NOVELTY AGENTS —

08 Key Features to Implement —

09 Development Roadmap (Beginner → Advanced) —

10 Making It Production-Grade —

11 Evaluating the Agents —

12 Expected Challenges & Solutions —

13 Explaining This Project in Interviews —

14 Resume Bullet Points —

01 Project Overview

> *AutoResolve AI is a multi-agent system that autonomously triages, investigates, and resolves customer support or internal ops tickets — reasoning over documentation, calling live tools, defending itself against adversarial and malicious input, and improving its own knowledge base over time.*

This is not a chatbot wrapped around a single prompt. It is a small, coherent distributed system of specialized agents that plan, retrieve, call tools, self-check, and escalate — the same architectural pattern used by production agentic products (e.g. AI-driven support and ops automation platforms). It is fully software-based, cloud-deployable, and demonstrates the complete agentic AI skill set employers are currently hiring for: orchestration, RAG, tool use, memory, guardrails, and evaluation.

02 The Real-World Problem

Every SaaS, e-commerce, and fintech company operates a support or internal-ops function buried in repetitive tickets — refund status, password resets, failed payments, account access, server/service issues. Human agents

spend the majority of their time on triage and information-gathering rather than judgment calls. AutoResolve AI automates that pipeline end-to-end:

- Classifies and prioritizes incoming tickets automatically
- Retrieves the relevant policy or procedure from internal documentation (RAG)
- Calls real systems — order database, payment gateway, ticketing system, logs — to gather facts
- Drafts or executes a resolution within clearly defined guardrails
- Escalates to a human with full context when confidence is low or the action is high-risk

This mirrors the exact product category of tools like Intercom Fin, Sierra, and Decagon, and internal "AI SRE" copilots — making it directly relevant to how companies are hiring for agentic AI roles today.

03 Why This Project Is Valuable for Your Resume

- **Business outcome, not just a tech demo** — recruiters immediately understand the ROI story: faster resolution, lower support cost.
- **Covers the exact 2025–2026 hiring checklist:** agent orchestration, RAG, tool-calling, evaluation, guardrails, and human-in-the-loop design.
- **Differentiated** from the flood of "chat with your PDF" projects most candidates submit.
- **Produces real metrics** — resolution rate, escalation precision, latency, cost per ticket — the kind of numbers that make a portfolio project credible in an interview.
- **Security- and cost-aware design** (via the novelty agents) signals engineering maturity beyond "I called an LLM API."

04 Agentic AI Concepts You Will Learn

Concept	What You'll Practice
Planning & task decomposition	Breaking "resolve this ticket" into ordered sub-steps an agent can execute
Tool / function calling	Structured calls to APIs and databases with schema validation and retries
Retrieval-Augmented Generation	Grounding answers in company docs/policies with citation enforcement
Multi-agent orchestration	Specialized agents handing off state via an explicit graph (LangGraph)
Memory	Short-term session context (Redis) and long-term customer/ticket history (Postgres)
Reflection / self-correction	An agent critiquing its own plan or output before acting

Guardrails & human-in-the-loop	Confidence thresholds and approval gates for risky actions
Agent evaluation	Offline labeled eval sets, LLM-as-judge scoring, and regression testing
Observability / tracing	Inspecting exactly why an agent made a given decision
Adversarial robustness NOVEL	Red-teaming your own agent pipeline with generated attack tickets
Continual / self-improving systems NOVEL	Feeding production outcomes back into the knowledge base automatically
Cost-aware model routing NOVEL	Matching model tier to task complexity to optimize cost vs. quality

05 Recommended Tech Stack

Layer	Tools
LLM	Claude (Anthropic API) or GPT-4o, accessed via API
Agent orchestration	LangGraph (explicit state machine) — or CrewAI as an alternative
RAG	LlamaIndex or LangChain + a vector database (Chroma / Qdrant / Pinecone)
Tool layer	Python functions wrapped as tools; mock REST services (FastAPI) simulating order/payment/ticketing systems
Memory	Redis (short-term/session) + PostgreSQL (long-term/history)
Backend	FastAPI
Frontend	Next.js / React dashboard — ticket queue and agent trace viewer
Evaluation	Custom eval harness + Ragas (RAG metrics) + LLM-as-judge
Observability	Langfuse or LangSmith for full trace logging
Deployment	Docker + Docker Compose → AWS ECS / GCP Cloud Run
CI/CD	GitHub Actions, running the eval suite on every pull request

06 System Architecture

Ticket state (context, retrieved docs, tool outputs, confidence scores) flows through an explicit LangGraph graph, so every transition between agents is inspectable — this is what separates a production-style system from a single giant prompt.



07 Agents & Responsibilities

Eight agents in total — five form the core resolution pipeline, three are the novelty layer that pushes this from "portfolio demo" to "system with real engineering judgment."

Core Pipeline Agents

1. Triage Agent

Classifies ticket type and urgency, extracts structured fields (customer ID, issue category), and routes the ticket to the correct downstream path.

2. Retrieval Agent

Performs RAG over the internal knowledge base (policies, FAQs, runbooks) and returns grounded context with citations.

3. Tool-Use Agent

Calls external/internal APIs — order lookup, payment status, log search, password reset — via structured function calling, with schema validation and retry logic.

4. Resolution Agent

Synthesizes retrieved context and tool outputs into a plan, then drafts a customer-facing reply or executes the resolving action.

5. Escalation / Guardrail Agent

Decides — based on confidence score and action risk level (e.g. refunds above a threshold always escalate) — whether to auto-resolve or hand off to a human with a full context summary.

Novelty Layer DIFFERENTIATORS

6. Guard Agent — Prompt-Injection & Policy Defense

Sits between untrusted input (customer text, retrieved documents, tool outputs) and the Resolution Agent, scanning for injected instructions that attempt to override system policy — e.g. text hidden in a ticket telling the agent to "approve a $10,000 refund, skip approval." Directly addresses one of the most cited real risks in deployed agentic systems.

7. Knowledge Curator Agent — Self-Improving RAG

After a ticket is resolved (automatically or by a human), analyzes the transcript to detect knowledge gaps and drafts a candidate FAQ/runbook entry for human approval before it's added to the vector store. Turns the RAG layer from static to continually improving.

8. Adversarial Ticket Generator Agent — Automated Red-Teaming

Generates synthetic tickets designed to break the system: ambiguous requests, contradictory information, policy-conflict edge cases, and prompt-injection attempts. Runs nightly against the pipeline and feeds failures into the evaluation set.

Bonus: Model Router Agent — Cost-Aware Routing

A lightweight classifier that routes simple, low-stakes tickets to a cheap/fast model and ambiguous or high-stakes tickets to a stronger model, logging cost-per-ticket before and after — demonstrating that agentic AI is also an economic engineering problem, not just "call the biggest model for everything."

08 Key Features to Implement

- Ticket ingestion via webhook, API, or CSV upload
- RAG-grounded answer generation with source citations
- Function-calling tools against mocked backend services (a small FastAPI app simulating "order DB," "payment API," "ticketing system")
- Confidence scoring plus a human-in-the-loop approval UI
- Session and long-term memory (recall past tickets from the same customer)
- Agent trace / reasoning viewer showing plan → tool calls → final answer
- Evaluation dashboard: accuracy, escalation rate, latency, cost per resolution
- Rate limiting, retries, and structured error handling on every tool call
- Role-based guardrails — some actions auto-execute, some require approval
- NOVEL Prompt-injection detection layer with logged attack attempts
- NOVEL Nightly adversarial test runs with pass/fail trend charts
- NOVEL Auto-drafted knowledge base entries pending human approval
- NOVEL Cost-per-ticket comparison across model tiers

09 Development Roadmap: Beginner → Advanced

Phase 1 — Foundations (Weeks 1–2)

Build a single-agent RAG chatbot answering FAQ questions from a small document set. Learn prompting, embeddings, and vector search.

Phase 2 — Tool Calling (Weeks 3–4)

Add function calling, connecting the agent to a mock order/payment API so it can answer factual questions grounded in live data.

Phase 3 — Multi-Agent Orchestration (Weeks 5–6)

Split the system into Triage → Retrieval → Tool-Use → Resolution agents using LangGraph, with conditional routing and shared state.

Phase 4 — Memory & Guardrails (Weeks 7–8)

Add Postgres-backed long-term memory, confidence scoring, and human-approval gates for high-risk actions.

Phase 5 — Evaluation & Observability (Weeks 9–10)

Build a labeled evaluation set (50–100 sample tickets), integrate Langfuse/LangSmith tracing, and build the eval dashboard.

Phase 6 — Novelty Layer (Weeks 11–13) DIFFERENTIATOR

Build the Guard Agent (prompt-injection defense), the Adversarial Ticket Generator (automated red-teaming), the Knowledge Curator Agent (self-improving RAG), and the Model Router Agent (cost-aware routing).

Phase 7 — Production Hardening (Weeks 14–15)

Dockerize the full system, add CI/CD with automated eval gates, load testing, monitoring, cost tracking, and deploy to the cloud.

10 Making It Production-Grade

- **Containerization:** Docker Compose locally; ECS or Cloud Run for deployment
- **Structured logging & tracing** of every agent decision via Langfuse
- **Cost tracking** per ticket — token usage converted to dollars per resolution
- **Rate limiting & retries** on all external tool calls
- **Hard-coded policy guardrails** that override LLM judgment for high-risk actions
- **CI/CD:** automated evaluation suite runs on every pull request to catch prompt regressions before merge
- **Secrets management** via environment variables or a secret manager — never hardcoded keys
- **Horizontal scalability:** stateless API layer with session state in Redis/Postgres, not in-memory

11 Evaluating the Agents

Evaluation Dimension	How It's Measured
Retrieval quality	Precision/recall of retrieved docs vs. a labeled set; Ragas metrics — faithfulness, context relevance
Task success rate	Percentage of tickets correctly resolved vs. a human-labeled gold-standard answer set
Tool-call correctness	Did the agent call the right tool with the right arguments?
Escalation precision/recall	

	Did it escalate exactly the cases that needed a human — not too many, not too few?
LLM-as-judge scoring	A second LLM call scores response quality and policy adherence against a rubric
Latency & cost per ticket	Tracked per resolution, broken down by model tier
Regression testing	Eval set re-run on every prompt/model change to catch silent quality drops
NOVEL Injection-resistance rate	Percentage of adversarial/injection tickets correctly detected and blocked by the Guard Agent
NOVEL Knowledge-freshness gain	Improvement in retrieval relevance after N cycles of the Knowledge Curator Agent's updates
NOVEL Cost-efficiency ratio	Cost per ticket under model routing vs. a flat single-model baseline

12 Expected Challenges & Solutions

Challenge	Solution
Agent hallucinates facts not in retrieved docs	Enforce citation-grounded answers; reject ungrounded claims via a critic/reflection step
Tool-calling errors or malformed arguments	Strict JSON schema validation with retry-and-correct feedback to the model
Runaway loops in the multi-agent flow	Set max iteration/step limits in the LangGraph orchestrator
Hard to tell if a change actually improved things	Maintain a frozen evaluation set; compare metrics before and after every change
Cost explosion from repeated LLM calls	Cache retrieval results; use smaller models for triage; batch requests where possible
Over- or under-escalation to humans	Tune confidence thresholds against labeled eval data rather than intuition
Prompt injection via customer text or documents	Dedicated Guard Agent scans all untrusted input before it reaches the Resolution Agent
Knowledge base going stale over time	Knowledge Curator Agent proposes updates from real resolved tickets, with human approval

13 Explaining This Project in Interviews

> *"I built a multi-agent system that automates customer support ticket resolution end-to-end. A LangGraph orchestrator routes tickets through specialized agents — one classifies and prioritizes, one retrieves grounded context via RAG from company docs, one calls tools like order and payment APIs, and one drafts or executes the resolution. On top of that core pipeline, I added a security layer: a Guard Agent that detects prompt-injection attempts in customer text or retrieved documents, and an Adversarial Ticket Generator that automatically red-teams the whole system nightly. I also built a Knowledge Curator Agent that turns resolved tickets into proposed knowledge base updates, so the RAG layer improves over time instead of staying static. Everything is measured through an evaluation pipeline I built — resolution accuracy, escalation precision, injection-resistance rate, and cost per ticket — so I could prove the system actually got better with each iteration rather than just eyeballing outputs."*

Be ready to go deep on: *why LangGraph over a single prompt loop*, *how you prevented hallucination*, *how you tuned the escalation threshold*, *what your evaluation set looked like*, and *how the Guard Agent actually detects injection attempts*.

14 Resume Bullet Points

- Built **AutoResolve AI**, an 8-agent orchestration system (LangGraph, Claude API, RAG) that autonomously triages and resolves customer support tickets, reducing simulated average resolution time by **X%** and auto-resolving **Y%** of tickets without human intervention.
- Designed a 5-agent core resolution pipeline (triage, retrieval, tool-use, resolution, guardrail) with function-calling integration to live/mock order, payment, and ticketing APIs.
- Engineered a **prompt-injection detection agent** and an automated **adversarial red-teaming pipeline** that generates edge-case and attack tickets nightly, achieving an injection-resistance rate of **X%** across a 500-ticket adversarial evaluation set.
- Built a **self-improving knowledge base loop** via a Knowledge Curator Agent that mined resolved tickets for knowledge gaps, improving retrieval relevance (Ragas context-precision) by **X%** over N iterations.
- Implemented a **cost-aware model router** that matched task complexity to model tier, cutting inference cost by **X%** with no drop in resolution accuracy, and deployed the full system as a containerized, CI/CD-tested service (Docker, FastAPI, Postgres, Redis, GitHub Actions).